UNIT V: Virtual Functions, Polymorphism & Exception Handling

Name:Kavitha B Reg no:192565041

Quick summary of Section A

Q	Bug	Class	Fix
1	Non-virtual base destructor (Line 7/41)	Undefined behavior, memory leak	`virtual ~Shape()`
2	Base catch before derived catch (Line 34/38)	Unreachable handler	Derived first, base next, `catch (...)` last
3	`throw e;` slices exception (Line 32)	Exception slicing	Bare `throw;`
4	Pure virtual `getPacketSize()` not overridden (Line 25)	Compile error: abstract class	Implement all pure virtuals
5	Polymorphic parameter by value (Line 17)	Object slicing, early binding	`const Entity&` parameter

Section A: Debugging Solutions

Each solution follows the required output format: (1) line number and bug classification, (2) theoretical analysis using vtable/vptr or exception-handling runtime semantics, (3) corrected C++ code, (4) dry run and terminal output. All buggy and corrected programs were compiled with g++ (C++17) and run; the outputs shown are the real terminal output.

Question 1: Non-Virtual Destructor Memory Leak

Item	Finding
Line / classification	**Line 7** (`~Shape()` declared without `virtual`) and **Line 41** (`delete shapePtr;`). Classification: **Runtime / Undefined-Behavior bug** (resource leak), caused by static binding of the destructor.
Buggy output	Shape created. Allocated 1000 vertices on heap. Rendering polygon with 1000 vertices. Shape destroyed. (ComplexPolygon destructor never runs)

Task 1: Why ~ComplexPolygon() is never called

The expression `*shapePtr` has static type `Shape`. Because `~Shape()` is **not virtual**, the compiler resolves the destructor call at compile time (static / early binding) using that static type, so it emits a direct call to `Shape::~Shape()`. No vtable lookup happens for the destructor, since the destructor has no vtable slot. `render()` is virtual, so it is dispatched through the vptr and works correctly, but the destructor is not. As a result `ComplexPolygon::~ComplexPolygon()` is skipped and `delete[] vertexArray` never executes, leaking 1000 × 2 × 8 = 16,000 bytes.

Task 2: Standard rule and behavior classification

C++ standard ([expr.delete]): if the static type of the object to be deleted is different from its dynamic type, the static type must be a base class of the dynamic type **and must have a virtual destructor**; otherwise the behavior is **undefined**. This is classified as **Undefined Behavior (UB)**. The program may leak (as here), crash,

or corrupt the heap, because `operator delete` may also be called with the wrong object size. g++ warns with `-Wdelete-non-virtual-dtor`.

Task 3: Fix and vtable explanation

Declare the base destructor `virtual ~Shape()`. The derived destructor is then automatically virtual (writing `override` is optional). Now each polymorphic class gets a destructor slot in its vtable:

- `Shape` vtable: `[ ~Shape, render ]` and `ComplexPolygon` vtable: `[ ~ComplexPolygon, render ]`.
- `delete shapePtr` loads the object's vptr (set by the most-derived constructor to the ComplexPolygon vtable), reads the destructor slot and calls `~ComplexPolygon()` (the "deleting destructor").
- `~ComplexPolygon()` runs its body (`delete[] vertexArray`), destroys its members, and then the compiler automatically chains into `~Shape()`. Destruction order is the reverse of construction: derived first, then base.
- Finally `operator delete` releases the object memory using the correct dynamic size.

Task 4: Corrected code and terminal output

Corrected code

```
#include <iostream>

class Shape {
public:
    Shape() { std::cout << "Shape created." << std::endl; }

    // FIX: destructor declared virtual
    virtual ~Shape() {
        std::cout << "Shape destroyed." << std::endl;
    }

    virtual void render() const {
        std::cout << "Rendering generic shape." << std::endl;
    }
};

class ComplexPolygon : public Shape {
private:
    double *vertexArray;
    int vertexCount;

public:
    ComplexPolygon(int count) : vertexCount(count) {
        vertexArray = new double[vertexCount * 2];
        std::cout << "Allocated " << vertexCount << " vertices on heap." << std::endl;
    }

    ~ComplexPolygon() {
        std::cout << "Deallocating polygon vertex heap memory." << std::endl;
        delete[] vertexArray; // Now executed
    }

    void render() const override {
        std::cout << "Rendering polygon with " << vertexCount << " vertices." << std::endl;
    }
};

int main() {
    Shape *shapePtr = new ComplexPolygon(1000);
    shapePtr->render();

    // Polymorphic deletion through base pointer (now safe)
    delete shapePtr;
```

```
    return 0;
}
```

Terminal output

```
Shape created.
Allocated 1000 vertices on heap.
Rendering polygon with 1000 vertices.
Deallocating polygon vertex heap memory.
Shape destroyed.
```

Step	Statement	Mechanism	Buggy	Fixed
1	new ComplexPolygon(1000)	Base constructor first, then derived; vptr ends at ComplexPolygon vtable	Shape created.	Shape created.
2	vertexArray = new double[2000]	Heap allocation (16,000 bytes)	Allocated 1000 vertices on heap.	Allocated 1000 vertices on heap.
3	shapePtr->render()	vptr → render slot → ComplexPolygon::render	Rendering polygon with 1000 vertices.	Rendering polygon with 1000 vertices.
4	delete shapePtr	Buggy: static call to ~Shape(). Fixed: vptr → destructor slot → ~ComplexPolygon()	(skipped)	Deallocating polygon vertex heap memory.
5	base destructor	Chained automatically after derived destructor	Shape destroyed. (array leaked)	Shape destroyed.

Question 2: Inverted Catch Handlers & Unreachable Code

Item	Finding
Line / classification	**Line 34** (base handler placed first) shadows **Line 38** (derived handler). Classification: **Logic bug / unreachable code** (compiler warning, not an error).
Buggy output	General Banking Error: Account overdrawn! (shortfall of $250 never reported)

Task 1: Compiler diagnostic on Line 38

g++ reports a **warning** (not an error), enabled by `-Wexceptions`:

```
warning: exception of type 'InsufficientFundsException' will be caught by earlier handler [-
Wexceptions]
note: for type 'BankingException'
```

Clang gives a similar warning ("exception of type ... will be caught by earlier handler"). The code compiles, so the bug is easy to miss unless warnings are enabled.

Task 2: Catch-matching mechanism

When `processWithdrawal` executes `throw InsufficientFundsException(...)`, the runtime creates the exception object (its dynamic type is `InsufficientFundsException`) and then searches the enclosing handlers **top to bottom in source order**. The **first** handler that can accept the object wins. A handler matches when its parameter type is the same as the thrown type **or a public, unambiguous base class** of it (a reference to base can bind to a derived object). `InsufficientFundsException` publicly inherits `BankingException`, so `catch (const BankingException &be)` matches. No later handler is considered, so Line 38 can never execute.

Task 3: Ordering rule

Order handlers from **most derived (most specific) to least derived (most general)**: subclass handlers before superclass handlers, with `catch (...)` always last. A handler for a base class placed earlier makes every handler for its derived classes unreachable.

Task 4: Corrected try-catch block

Corrected code

```
#include <iostream>
#include <string>

// Base custom exception
class BankingException {
protected:
    std::string msg;
public:
    BankingException(std::string m) : msg(m) {}
    virtual std::string getMessage() const { return msg; }
};

// Derived specific exception
class InsufficientFundsException : public BankingException {
private:
    double shortfall;
public:
    InsufficientFundsException(std::string m, double sf)
        : BankingException(m), shortfall(sf) {}

    double getShortfall() const { return shortfall; }
};

void processWithdrawal(double balance, double amount) {
    if (amount > balance) {
        throw InsufficientFundsException("Account overdrawn!", amount - balance);
    }
}

int main() {
    try {
        processWithdrawal(500.0, 750.0);
    }
    // FIX: most-derived handler first, base next, catch-all last
    catch (const InsufficientFundsException &ife) {
        std::cout << "Shortfall of: $" << ife.getShortfall() << std::endl;
    }
    catch (const BankingException &be) {
        std::cout << "General Banking Error: " << be.getMessage() << std::endl;
    }
    catch (...) {
        std::cout << "Unknown error caught by fallback handler." << std::endl;
    }

    return 0;
}
```

Terminal output

```
Shortfall of: $250
```

Step	Event	Handler test order	Result
1	processWithdrawal(500.0, 750.0): 750 > 500	-	Throws InsufficientFundsException("Account overdrawn!", 250)

Step	Event	Handler test order	Result
2	Stack unwinds to main	1) catch InsufficientFundsException&	Exact match, handler runs
3	Handler body	(BankingException& and catch(...) skipped)	Prints "Shortfall of: $250"
4	After catch	-	Control continues after try-catch; return 0

Question 3: Rethrow Slicing Bug (throw e; vs. throw;)

Item	Finding
Line / classification	**Line 32** (`throw e;`). Classification: **Exception slicing / incorrect rethrow** (logic bug).
Buggy output	[AUDIT LOG]: Intercepted DB error. Client caught generic DatabaseError: Query timed out connecting to shard 3

Task 1: Current output and why the TimeoutError handler is bypassed

The program prints the two lines shown above. Inside `intermediateAuditor`, the caught variable `e` has static type `const DatabaseError&`. `throw e;` does not rethrow the active exception; it **throws a new exception object, copy-initialised from the static type** `DatabaseError`. The copy constructor of `DatabaseError` copies only the base subobject, so the `TimeoutError` part (`timeoutMs`) is sliced off, and the original `TimeoutError` is destroyed when the handler exits. In `main`, the thrown object's type is now plain `DatabaseError`. A handler for `TimeoutError&` cannot bind to a base-class object, so it is skipped, and `catch (const DatabaseError &de)` matches instead.

Task 2: throw e; vs. bare throw;

	throw e;	throw; (bare)
What is thrown	A NEW exception object	The SAME, currently handled exception object
Type used	Static type of `e` (here DatabaseError)	Dynamic type (here TimeoutError)
Copy construction	Yes: copy constructor of the static type runs, causing slicing	None: the original object is propagated by reference
Derived data	Lost (`timeoutMs` gone)	Preserved
Cost	Extra copy, and the original is destroyed	No extra allocation or copy

Task 3: Correction of Line 32

Replace `throw e;` with the bare `throw;`. It may only be used inside a handler (or a function called from one) while an exception is being handled.

Corrected lines

```
catch (const DatabaseError &e) {
    std::cout << "[AUDIT LOG]: Intercepted DB error." << std::endl;
    throw;   // rethrow the ORIGINAL TimeoutError object, no slicing
}
```

Full corrected program

```
#include <iostream>
#include <exception>
#include <string>
```

```
class DatabaseError : public std::exception {
protected:
    std::string errorDetails;
public:
    DatabaseError(std::string msg) : errorDetails(msg) {}
    const char* what() const noexcept override {
        return errorDetails.c_str();
    }
};

class TimeoutError : public DatabaseError {
public:
    int timeoutMs;
    TimeoutError(std::string msg, int ms)
        : DatabaseError(msg), timeoutMs(ms) {}
};

void executeQuery() {
    throw TimeoutError("Query timed out connecting to shard 3", 5000);
}

void intermediateAuditor() {
    try {
        executeQuery();
    }
    catch (const DatabaseError &e) {
        std::cout << "[AUDIT LOG]: Intercepted DB error." << std::endl;

        // FIX: bare throw rethrows the ORIGINAL TimeoutError object
        throw;
    }
}

int main() {
    try {
        intermediateAuditor();
    }
    catch (const TimeoutError &te) {
        std::cout << "Client caught TimeoutError! Duration: " << te.timeoutMs << " ms" << std::endl;
    }
    catch (const DatabaseError &de) {
        std::cout << "Client caught generic DatabaseError: " << de.what() << std::endl;
    }
    return 0;
}
```

Task 4: Corrected output

```
[AUDIT LOG]: Intercepted DB error.
Client caught TimeoutError! Duration: 5000 ms
```

Step	Event	Buggy (throw e;)	Fixed (throw;)
1	executeQuery() throws	TimeoutError(msg, 5000) created	Same
2	Caught in intermediateAuditor by DatabaseError&	Prints [AUDIT LOG]	Prints [AUDIT LOG]
3	Rethrow	New DatabaseError copy (sliced); TimeoutError destroyed	Original TimeoutError propagates
4	main handler search	TimeoutError& no match; DatabaseError& matches	TimeoutError& matches
5	Printed	Client caught generic DatabaseError: ...	Client caught TimeoutError! Duration: 5000 ms

Question 4: Instantiating an Incomplete Abstract Class

Item	Finding
Line / classification	**Line 25** (`VideoPacket packet(1042);`), caused by the missing override at the end of `VideoPacket`. Classification: **Compile-time error** (abstract class instantiation).

Task 1: Exact compilation error

```
error: cannot declare variable 'packet' to be of abstract type 'VideoPacket'
note: because the following virtual functions are pure within 'VideoPacket':
note:     'virtual int NetworkPacket::getPacketSize()'
```

Other compilers: Clang says "variable type 'VideoPacket' is an abstract class"; MSVC gives error C2259 "'VideoPacket': cannot instantiate abstract class".

Task 2: What makes a class abstract

A class is **abstract** if it declares or **inherits** at least one pure virtual function (`virtual ... = 0;`) that has not been given a final overrider. An abstract class **cannot be instantiated directly** (no objects of that exact type), but it can be used as a base class and through pointers and references to it, which is how runtime polymorphism is achieved. Its constructors run only as part of a derived object.

Task 3: When a derived class becomes concrete

A derived class becomes concrete when **every** pure virtual function inherited from all its bases has a non-pure override, either in the class itself or in an intermediate base. Here `VideoPacket` overrides `serialize()` but not `getPacketSize()`, so it stays abstract. (A pure virtual function may be overridden by another pure declaration, which keeps the class abstract.)

Task 4: Corrected code

Corrected code (adds the missing override)

```
#include <iostream>

class NetworkPacket {
public:
    virtual ~NetworkPacket() {}

    // Pure Virtual Functions
    virtual void serialize() = 0;
    virtual int getPacketSize() = 0;
};

// Subclass now overrides BOTH pure virtual functions
class VideoPacket : public NetworkPacket {
private:
    int frameID;

public:
    VideoPacket(int id) : frameID(id) {}

    void serialize() override {
        std::cout << "Serializing H.264 Video Frame: " << frameID << std::endl;
    }
    int getPacketSize() override {
        return 1500; // bytes (example payload size)
    }
};

int main() {
```

```
    VideoPacket packet(1042); // OK: VideoPacket is now concrete
    packet.serialize();
    std::cout << "Packet size: " << packet.getPacketSize() << " bytes" << std::endl;

    // Runtime polymorphism through the abstract interface
    NetworkPacket *p = new VideoPacket(2001);
    p->serialize();
    delete p;
    return 0;
}
```

Terminal output

```
Serializing H.264 Video Frame: 1042
Packet size: 1500 bytes
Serializing H.264 Video Frame: 2001
```

The `NetworkPacket*` section shows the payoff: `p->serialize()` is dispatched through the vptr of the `VideoPacket` object, and `delete p` is safe because `~NetworkPacket()` is virtual.

Question 5: Pass-by-Value Slicing Suppresses Dynamic Binding

Item	Finding
Line / classification	**Line 17** (`void triggerEncounter(Entity e)`). Classification: **Object slicing / early binding** (logic bug).
Buggy output	Entity performs generic scratch.

Task 1: Why "Entity performs generic scratch." is printed

Passing `boss` (a `Dragon`) to a parameter of type `Entity` by value **copy-initialises a brand-new `Entity` object** from the `Entity` part of `boss`. The `Dragon`-specific part is **sliced off**. Inside the function, `e` is a genuine, complete `Entity` whose static and dynamic types are both `Entity`, so `e.attack()` calls `Entity::attack()`. The original `boss` is untouched; it is simply not the object being used.

Task 2: Role of the vptr: by value vs. by reference

- **By value:** The vptr is **not copied**. The `Entity` copy constructor, like any constructor, sets the new object's vptr to the vtable of the class being constructed, i.e. the **Entity vtable**. So the vptr inside the copied `e` points to `Entity`'s vtable, and the `attack` slot there holds `Entity::attack`. Late binding still works technically, but only for the wrong type.
- **By reference (or pointer):** No new object is made. The reference aliases `boss`, whose vptr (set by `Dragon`'s constructor) points to the **Dragon vtable**. `e.attack()` loads that vptr, indexes the `attack` slot and calls `Dragon::attack()`: dynamic dispatch.

Task 3: Refactor for late binding

Take the parameter by `const Entity&` (or by `const Entity*`). Since `attack()` is a `const` member, a const reference is sufficient.

Corrected code

```
#include <iostream>

class Entity {
public:
    virtual ~Entity() {}
    virtual void attack() const {
```

```
        std::cout << "Entity performs generic scratch." << std::endl;
    }
};

class Dragon : public Entity {
public:
    void attack() const override {
        std::cout << "Dragon breathes scorching fire!" << std::endl;
    }
};

// FIX: take parameter by const reference -> no slicing, vptr preserved
void triggerEncounter(const Entity &e) {
    e.attack(); // Late binding: Dragon::attack()
}

int main() {
    Dragon boss;

    triggerEncounter(boss);

    return 0;
}
```

Task 4: Verified terminal output

```
Dragon breathes scorching fire!
```

Step	Call	Object used inside function	vptr points to	Output
Buggy	triggerEncounter(boss)	Sliced copy `Entity e`	Entity vtable	Entity performs generic scratch.
Fixed	triggerEncounter(boss)	Alias of `boss` (`const Entity&`)	Dragon vtable	Dragon breathes scorching fire!

Section B: Pseudo-Code Solutions

All pseudo-code uses the prescribed notation: ABSTRACT CLASS, PURE VIRTUAL METHOD, OVERRIDE METHOD, VIRTUAL DESTRUCTOR(), TRY, THROW NEW, CATCH (e AS Type), RETHROW and CATCH (...). Each answer ends with a trace table for traceability.

Question 1: Universal Payment Gateway

Task 1: Structured pseudo-code

Pseudo-code

```
// ===== Abstract interface =====
ABSTRACT CLASS PaymentGateway
    PURE VIRTUAL METHOD Authorize(amount AS REAL) RETURNS BOOLEAN
    PURE VIRTUAL METHOD Capture() RETURNS BOOLEAN
    VIRTUAL DESTRUCTOR()
        PRINT "PaymentGateway released"
    END DESTRUCTOR
END CLASS

// ===== Concrete gateway 1 =====
CLASS CreditCardGateway INHERITS PaymentGateway
    PRIVATE cardNumber AS STRING
    PRIVATE cvv AS STRING
    PRIVATE authorizedAmount AS REAL <- 0.0
```

```
    CONSTRUCTOR(card AS STRING, code AS STRING)
        cardNumber <- card
        cvv <- code
    END CONSTRUCTOR

    OVERRIDE METHOD Authorize(amount AS REAL) RETURNS BOOLEAN
        IF LENGTH(cardNumber) != 16 OR NOT IS_ALL_DIGITS(cardNumber) THEN
            PRINT "Card rejected: number must be 16 digits"
            RETURN FALSE
        END IF
        IF LENGTH(cvv) != 3 OR NOT IS_ALL_DIGITS(cvv) THEN
            PRINT "Card rejected: CVV must be 3 digits"
            RETURN FALSE
        END IF
        IF amount <= 0 THEN
            RETURN FALSE
        END IF
        authorizedAmount <- amount
        PRINT "Card authorized: ", amount
        RETURN TRUE
    END METHOD

    OVERRIDE METHOD Capture() RETURNS BOOLEAN
        IF authorizedAmount = 0 THEN
            RETURN FALSE
        END IF
        PRINT "Card captured: ", authorizedAmount
        authorizedAmount <- 0.0
        RETURN TRUE
    END METHOD

    VIRTUAL DESTRUCTOR()
        PRINT "CreditCardGateway released"
    END DESTRUCTOR
END CLASS

// ===== Concrete gateway 2 =====
CLASS CryptoGateway INHERITS PaymentGateway
    PRIVATE walletHash AS STRING
    PRIVATE walletBalance AS REAL
    PRIVATE heldTotal AS REAL <- 0.0
    CONSTANT GAS_RATE AS REAL <- 0.01          // simulated gas fee = 1% of amount

    CONSTRUCTOR(hash AS STRING, balance AS REAL)
        walletHash <- hash
        walletBalance <- balance
    END CONSTRUCTOR

    OVERRIDE METHOD Authorize(amount AS REAL) RETURNS BOOLEAN
        IF NOT STARTS_WITH(walletHash, "0x") THEN
            PRINT "Wallet rejected: hash must start with 0x"
            RETURN FALSE
        END IF
        gasFee <- amount * GAS_RATE
        IF walletBalance < amount + gasFee THEN
            PRINT "Rejected: balance cannot cover amount + gas fee"
            RETURN FALSE
        END IF
        heldTotal <- amount + gasFee
        PRINT "Crypto authorized: ", amount, " + gas ", gasFee
        RETURN TRUE
    END METHOD

    OVERRIDE METHOD Capture() RETURNS BOOLEAN
        IF heldTotal = 0 THEN
            RETURN FALSE
        END IF
        walletBalance <- walletBalance - heldTotal
```

```
        PRINT "Crypto captured: ", heldTotal
        heldTotal <- 0.0
        RETURN TRUE
    END METHOD

    VIRTUAL DESTRUCTOR()
        PRINT "CryptoGateway released"
    END DESTRUCTOR
END CLASS

// ===== Driver =====
FUNCTION Main()
    DECLARE gatewayPtr AS POINTER TO PaymentGateway
    READ choice

    IF choice = "CARD" THEN
        gatewayPtr <- NEW CreditCardGateway("4111111111111111", "123")
    ELSE IF choice = "CRYPTO" THEN
        gatewayPtr <- NEW CryptoGateway("0xA91F3C7D", 2000.00)
    ELSE
        PRINT "Unknown gateway"
        RETURN
    END IF

    IF gatewayPtr->Authorize(1500.00) THEN          // late binding via vtable
        IF gatewayPtr->Capture() THEN                // late binding via vtable
            PRINT "Payment of 1500.00 completed"
        END IF
    ELSE
        PRINT "Payment declined"
    END IF

    DEALLOCATE gatewayPtr                           // safe: virtual destructor
END FUNCTION
```

Task 2: Trace for a $1,500 cryptocurrency payment

Assumptions: `choice = "CRYPTO"`, wallet hash `0xA91F3C7D`, wallet balance 2000.00, gas fee 1% = 15.00.

Step	Statement executed	Dispatch through base pointer	State / Output
1	READ choice	-	choice = "CRYPTO"
2	gatewayPtr <- NEW CryptoGateway(...)	PaymentGateway constructor, then CryptoGateway constructor; vptr → CryptoGateway vtable	gatewayPtr static type PaymentGateway*, dynamic type CryptoGateway
3	gatewayPtr->Authorize(1500.00)	vptr → vtable[Authorize] → CryptoGateway::Authorize	Prefix "0x" OK; gasFee = 15.00; 2000.00 >= 1515.00; heldTotal = 1515.00 PRINT Crypto authorized: 1500.00 + gas 15.00 returns TRUE
4	gatewayPtr->Capture()	vptr → vtable[Capture] → CryptoGateway::Capture	walletBalance = 2000.00 − 1515.00 = 485.00 PRINT Crypto captured: 1515.00 returns TRUE
5	PRINT "Payment of 1500.00 completed"	-	Payment of 1500.00 completed
6	DEALLOCATE gatewayPtr	vptr → vtable[destructor] → CryptoGateway destructor	PRINT CryptoGateway released
7	(automatic base chaining)	~PaymentGateway() runs after derived destructor	PRINT PaymentGateway released; memory freed

Question 2: Resilient Banking Engine (Nested Try-Catch & Rethrowing)

Task 1: Structured pseudo-code

Pseudo-code

```
// ===== Exception hierarchy =====
CLASS TransactionException
    PROTECTED reason AS STRING
    CONSTRUCTOR(r AS STRING)
        reason <- r
    END CONSTRUCTOR
    METHOD GetReason() RETURNS STRING
        RETURN reason
    END METHOD
    VIRTUAL DESTRUCTOR()
    END DESTRUCTOR
END CLASS

CLASS AccountFrozenException INHERITS TransactionException
    CONSTRUCTOR(r AS STRING)
        CALL TransactionException(r)
    END CONSTRUCTOR
END CLASS

CLASS DailyLimitExceededException INHERITS TransactionException
    PRIVATE excessAmount AS REAL
    CONSTRUCTOR(r AS STRING, excess AS REAL)
        CALL TransactionException(r)
        excessAmount <- excess
    END CONSTRUCTOR
    METHOD GetExcess() RETURNS REAL
        RETURN excessAmount
    END METHOD
END CLASS

CONSTANT DAILY_LIMIT AS REAL <- 50000.00

// ===== Tier 1: low-level =====
FUNCTION DebitLedger(account AS Account, amount AS REAL)
    IF account.isFrozen THEN
        THROW NEW AccountFrozenException("Account " + account.id + " is frozen")
    END IF
    IF account.debitedToday + amount > DAILY_LIMIT THEN
        excess <- account.debitedToday + amount - DAILY_LIMIT
        THROW NEW DailyLimitExceededException("Daily limit of 50000 exceeded", excess)
    END IF
    account.balance <- account.balance - amount
    account.debitedToday <- account.debitedToday + amount
END FUNCTION

// ===== Tier 2: intermediate auditor =====
FUNCTION AuditTransactionWrapper(account AS Account, amount AS REAL)
    TRY
        CALL DebitLedger(account, amount)
        PRINT "Ledger debit OK"
    CATCH (e AS TransactionException)          // by reference: matches base and all derived
        WRITE AUDIT LOG "SECURITY AUDIT: " + e.GetReason()
                        + " | acct=" + account.id + " | amt=" + amount
        RETHROW                                // same object, type preserved
    END TRY
END FUNCTION

// ===== Tier 3: high-level client =====
FUNCTION Main()
    account <- NEW Account("AC-1001", 20000.00, TRUE, 0.0)   // id, balance, isFrozen, debitedToday
    TRY
        CALL AuditTransactionWrapper(account, 1000.00)
        PRINT "Withdrawal successful"
    CATCH (e AS AccountFrozenException)        // most derived first
```

```
        PRINT "Notice: your account is frozen. Please contact the bank."
    CATCH (e AS DailyLimitExceededException)
        PRINT "Notice: daily limit exceeded by ", e.GetExcess()
    CATCH (e AS TransactionException)          // general base fallback
        PRINT "Transaction failed: ", e.GetReason()
    CATCH (...)                                // universal fallback, always last
        PRINT "Unexpected system error"
    END TRY
END FUNCTION
```

Task 2: Dry-run trace, $1,000 withdrawal on a frozen account

Step	Location	Event / control jump	Output / state
1	Main	Enters TRY block; calls AuditTransactionWrapper(account, 1000.00)	account.isFrozen = TRUE
2	AuditTransactionWrapper	Enters its own (nested) TRY; calls DebitLedger	-
3	DebitLedger	IF account.isFrozen is TRUE: THROW NEW AccountFrozenException("Account AC-1001 is frozen")	Exception object created (dynamic type AccountFrozenException)
4	DebitLedger	Stack unwinding: frame of DebitLedger is exited, rest of function and "Ledger debit OK" skipped	Control jumps to nearest enclosing handler
5	AuditTransactionWrapper	CATCH (e AS TransactionException) matches (AccountFrozenException is-a TransactionException)	-
6	AuditTransactionWrapper	WRITE AUDIT LOG	SECURITY AUDIT: Account AC-1001 is frozen | acct=AC-1001 | amt=1000.00
7	AuditTransactionWrapper	RETHROW: same exception object propagates, frame exited (second unwinding)	Dynamic type still AccountFrozenException
8	Main	Handlers tested in order: AccountFrozenException first, exact match	Handler selected
9	Main	Handler body executes; remaining catches skipped; "Withdrawal successful" never printed	Notice: your account is frozen. Please contact the bank.
10	Main	Control continues after END TRY; balance unchanged	account.balance = 20000.00

Question 3: Polymorphic Game Engine with Virtual Destructors

Task 1: Structured pseudo-code

Pseudo-code

```
ABSTRACT CLASS GameObject
    PURE VIRTUAL METHOD Update(deltaTime AS REAL)
    PURE VIRTUAL METHOD Render()
    VIRTUAL DESTRUCTOR()
        PRINT "~GameObject: base cleanup"
    END DESTRUCTOR
END CLASS

CLASS ParticleSystem INHERITS GameObject
    PRIVATE particles AS POINTER TO REAL ARRAY     // heap memory
    PRIVATE particleCount AS INTEGER

    CONSTRUCTOR(count AS INTEGER)
        particleCount <- count
        particles <- NEW REAL ARRAY[count * 2]      // x and y per particle
        PRINT "ParticleSystem created with ", count, " particles"
    END CONSTRUCTOR
```

```
    OVERRIDE METHOD Update(deltaTime AS REAL)
        FOR i FROM 0 TO particleCount * 2 - 1
            particles[i] <- particles[i] + deltaTime * VELOCITY
        END FOR
    END METHOD

    OVERRIDE METHOD Render()
        PRINT "Rendering ", particleCount, " particles"
    END METHOD

    VIRTUAL DESTRUCTOR()
        PRINT "~ParticleSystem: releasing particle array"
        DEALLOCATE ARRAY particles
    END DESTRUCTOR
END CLASS

CLASS Player INHERITS GameObject
    OVERRIDE METHOD Update(deltaTime AS REAL)
        PRINT "Player moves"
    END METHOD
    OVERRIDE METHOD Render()
        PRINT "Rendering player"
    END METHOD
    VIRTUAL DESTRUCTOR()
        PRINT "~Player: cleanup"
    END DESTRUCTOR
END CLASS

FUNCTION Main()
    DECLARE sceneEntities AS ARRAY[2] OF POINTER TO GameObject
    sceneEntities[0] <- NEW ParticleSystem(500)
    sceneEntities[1] <- NEW Player()

    // One simulation step
    FOR i FROM 0 TO 1
        sceneEntities[i]->Update(0.016)             // late binding
        sceneEntities[i]->Render()                  // late binding
    END FOR

    // Leak-free cleanup
    FOR i FROM 0 TO 1
        DEALLOCATE sceneEntities[i]                 // relies on virtual destructors
    END FOR
END FUNCTION
```

Task 2: Cleanup-phase trace

Step	Statement	What the runtime does	Output / effect
1	DEALLOCATE sceneEntities[0] (static type GameObject*, dynamic type ParticleSystem)	Reads vptr of the object, which points to the ParticleSystem vtable; calls the destructor slot → ParticleSystem destructor (not GameObject's)	-
2	Body of ~ParticleSystem()	Derived destructor runs FIRST	PRINT ~ParticleSystem: releasing particle array Heap array of 1000 reals freed
3	Member cleanup	Data members of ParticleSystem destroyed	-
4	Base destructor chaining	Compiler automatically calls ~GameObject() AFTER the derived destructor	PRINT ~GameObject: base cleanup
5	Memory release	Object memory returned using correct dynamic size	No leak

Step	Statement	What the runtime does	Output / effect
6	DEALLOCATE sceneEntities[1] (dynamic type Player)	vptr → Player vtable → ~Player(), then ~GameObject()	PRINT ~Player: cleanup PRINT ~GameObject: base cleanup

Destruction order rule: most-derived destructor first, then each base in reverse order of construction (construction order is base first, derived last). Without VIRTUAL on `~GameObject()`, step 1 would call only `~GameObject()`, the particle array would leak, and the behavior would be undefined.

Question 4: File Stream Pipeline with Stack Unwinding & Catch-All

Task 1: Structured pseudo-code (RAII and stack unwinding)

Pseudo-code

```
CLASS ResourceGuard
    PRIVATE handle AS FILE_HANDLE

    CONSTRUCTOR(fileName AS STRING)
        handle <- ACQUIRE_HANDLE(fileName)          // resource acquired on construction
        PRINT "ResourceGuard: handle acquired for ", fileName
    END CONSTRUCTOR

    METHOD ReadBlock() RETURNS BYTES
    METHOD EndOfStream() RETURNS BOOLEAN
    METHOD ReadStoredChecksum() RETURNS INTEGER

    DESTRUCTOR()
        RELEASE_HANDLE(handle)                      // resource released automatically
        PRINT "ResourceGuard: handle released"
    END DESTRUCTOR
END CLASS

CLASS CRCValidationException
    PUBLIC expected AS INTEGER
    PUBLIC calculated AS INTEGER
    CONSTRUCTOR(exp AS INTEGER, calc AS INTEGER)
        expected <- exp
        calculated <- calc
    END CONSTRUCTOR
END CLASS

FUNCTION IngestDataStream(fileName AS STRING)
    DECLARE guard AS ResourceGuard(fileName)        // automatic (stack) object
    DECLARE calculated AS INTEGER <- 0

    WHILE NOT guard.EndOfStream()
        block <- guard.ReadBlock()
        calculated <- UPDATE_CRC(calculated, block)
    END WHILE

    expected <- guard.ReadStoredChecksum()
    IF calculated != expected THEN
        THROW NEW CRCValidationException(expected, calculated)
    END IF

    PRINT "Ingestion complete"
END FUNCTION                                        // normal exit also destroys guard

FUNCTION Main()
    TRY
        CALL IngestDataStream("telemetry.bin")
        PRINT "Pipeline finished OK"
    CATCH (e AS CRCValidationException)             // specific handler first
        PRINT "CRC mismatch: expected ", HEX(e.expected), " calculated ", HEX(e.calculated)
```

```
    CATCH (...)                                    // universal fallback, always last
        PRINT "Unknown failure in ingestion pipeline"
    END TRY
END FUNCTION
```

Task 2: Event sequence when a CRC error occurs

Assumed data: stored (expected) CRC = 0xA1B2C3D4, computed CRC = 0x7F00D1E2.

Step	Phase	Event	Output / state
1	Normal execution	Main enters TRY, calls IngestDataStream("telemetry.bin")	-
2	Normal execution	Local `guard` constructed on the stack, handle acquired	ResourceGuard: handle acquired for telemetry.bin
3	Normal execution	Loop reads all blocks, calculated = 0x7F00D1E2; expected read as 0xA1B2C3D4	-
4	Throw event	calculated != expected: THROW NEW CRCValidationException(0xA1B2C3D4, 0x7F00D1E2)	Exception object created; remaining statements skipped
5	Stack unwinding	Runtime leaves IngestDataStream and destroys its automatic objects: `guard` destructor runs	ResourceGuard: handle released
6	Handler search	Back in Main: CATCH (e AS CRCValidationException) matches exactly; catch (...) not reached	-
7	Catch execution	Handler body runs AFTER the guard was already destroyed	CRC mismatch: expected A1B2C3D4 calculated 7F00D1E2
8	After catch	Exception object destroyed; control continues after END TRY	"Pipeline finished OK" is skipped

The key point: **step 5 happens before step 7**. The handle is released automatically during stack unwinding even though no cleanup code was written in the catch block. This is RAII: resource lifetime is tied to object lifetime. If an unexpected exception type were thrown, `catch (...)` would handle it, and the guard would be released in the same way.

Question 5: Telemetry Calibration with noexcept Contracts

Task 1: Structured pseudo-code

Pseudo-code

```
CLASS OutOfBoundsException
    PUBLIC value AS REAL
    CONSTRUCTOR(v AS REAL)
        value <- v
    END CONSTRUCTOR
END CLASS

CONSTANT MIN_TEMP AS REAL <- -273.15
CONSTANT MAX_TEMP AS REAL <- 1500.0

// Potentially throwing function: noexcept(FALSE)
FUNCTION CalibrateThermalSensors(rawVoltage AS REAL) RETURNS REAL  NOEXCEPT(FALSE)
    temperature <- rawVoltage * 100.0 - 273.15       // assumed linear sensor calibration
    IF temperature < MIN_TEMP OR temperature > MAX_TEMP THEN
        THROW NEW OutOfBoundsException(temperature)
    END IF
    RETURN temperature
END FUNCTION

// Strict non-throwing contract
```

```
FUNCTION CommitTelemetry(finalTemp AS REAL)  NOEXCEPT
    TRY
        WRITE finalTemp TO avionicsBus
        PRINT "Committed: ", finalTemp
    CATCH (...)                                     // suppress everything
        SET busFaultFlag <- TRUE                    // record failure without throwing
    END TRY
END FUNCTION        // an escaping exception would call terminate()

FUNCTION Main()
    DECLARE rawVoltage AS REAL
    READ rawVoltage
    TRY
        temp <- CalibrateThermalSensors(rawVoltage)  // may throw
        CALL CommitTelemetry(temp)                   // guaranteed not to throw
    CATCH (e AS OutOfBoundsException)
        PRINT "Calibration fault: reading ", e.value, " outside [-273.15, 1500.0]"
        PRINT "Telemetry commit skipped; entering safe mode"
    CATCH (...)
        PRINT "Unknown calibration failure"
    END TRY
END FUNCTION
```

Task 2: Trace with a faulty reading of -300.0 °C

Input: rawVoltage = -0.2685 V, so temperature = (-0.2685 × 100.0) − 273.15 = -26.85 − 273.15 = **-300.0 °C**.

Step	Location	Event / control jump	Output / state
1	Main	READ rawVoltage; enters TRY	rawVoltage = -0.2685
2	Main	Calls CalibrateThermalSensors(-0.2685)	noexcept(FALSE): may throw
3	CalibrateThermalSensors	temperature = -300.0	-
4	CalibrateThermalSensors	Bounds check: -300.0 < MIN_TEMP (-273.15) is TRUE; THROW NEW OutOfBoundsException(-300.0)	Exception created; RETURN temperature not executed
5	Stack unwinding	CalibrateThermalSensors frame exited; control jumps to Main's handlers	-
6	Main	CATCH (e AS OutOfBoundsException) matches; CATCH (...) not reached	-
7	Main	Handler body executes. CommitTelemetry is NEVER called, so the bad value never reaches the avionics bus (the line after the throwing call was skipped)	Calibration fault: reading -300.0 outside [-273.15, 1500.0] Telemetry commit skipped; entering safe mode
8	Main	Control continues after END TRY; program ends normally	No terminate(): the noexcept function was never involved in the throw

Contract note: with a valid reading, `CommitTelemetry` runs and any internal failure is swallowed by its own `CATCH (...)`, so nothing can escape the NOEXCEPT boundary. If something did escape, the runtime would call `std::terminate()` immediately, with no stack unwinding guarantee, which in this system means an abort.